HDOC DAY 64

Find the Duplicate Element Using XOR

Aim Find the single repeated element in a variable-length integer array using pointers and one traversal.

Algorithm

1. Read the size n (at least 2) and the n array elements.

2. Assume values are 1 through n-1, with exactly one value repeated once.

3. Initialize result to zero; XOR result with every array element and with integers 1 through n-1.

4. Equal values cancel under XOR, leaving the duplicate. Print it.

C Program

```
#include <stdio.h>

void duplicate_element(const int *nums, int size)
{
    int result = 0;
    for (int i = 0; i < size; i++)
        result ^= *(nums + i);
    for (int i = 1; i < size; i++)
        result ^= i;
    printf("Repeated element: %d\n", result);
}

int main(void)
{
    int size;
    printf("Enter size: ");
    if (scanf("%d", &size) != 1 || size < 2) {
        printf("Invalid size.\n");
        return 1;
    }
    int nums[size];
    printf("Enter %d elements: ", size);
    for (int i = 0; i < size; i++)
        if (scanf("%d", &nums[i]) != 1) return 1;
    duplicate_element(nums, size);
    return 0;
}
```

Test Cases

Input	Expected result
1 3 4 2 2	2
3 1 3 4 2	3
1 4 5 2 3 4	4

Compilation and Execution (Mac Terminal)

```
clang -std=c99 -Wall -Wextra day64.c -o day64
./day64
```

Sample Output

```
Enter size: 5
Enter 5 elements: 1 3 4 2 2
```

```
Repeated element: 2
```

Important Note

The XOR method requires the complete expected set 1, 2, ..., size-1, with one repeated value. The prompt examples follow this pattern; without the expected range, XOR alone cannot identify an arbitrary duplicate.

Result

The program was implemented using the required technique and produces the expected results for the given test cases.